The object referenced by `SteamShovel.prototype` is the prototypical steam shovel. The `speak` method is created by assigning it as a property on the prototypical steam shovel. You can tell a constructor function which object to use as the prototype for future objects:

```
SteamShovel.prototype = {}
```

CoffeeScript classes are syntactic sugar for prototypes and constructor functions. This means you can change properties on the prototype:

```
SteamShovel.prototype.grumpy = yes
```

Now all of the objects created using the `SteamShovel` constructor have a `grumpy` property, inherited via the prototype chain:

```
gus.grumpy
# true
```

This turns out to be useful for dynamically modifying objects and classes.

### 5.5.3. Dynamic classes

By adding a property to the prototype, you're adding it to all the objects that use it *without touching the class declaration*:

```
class Example
example = new Example
Example.prototype.justAdded = -> "just added!"

example.justAdded()
# "just added!"
```

You probably remember doing something similar with raw objects in section 4.6 using `Object.create`. Now you're doing exactly the same thing by accessing the prototype for a whole class of objects by modifying the `prototype` object via the class.

#### Prototype shorthand